

ФСП спец. трек — рабочее ТЗ и разделение по команде

Продукт: веб-платформа подбора ИТ-специалистов с обратной механикой (работодатель сам находит категорию и приглашает кандидата).

Цель хакатона: показать сквозной сценарий и выиграть по двум главным критериям — механика подбора (35%) и тестирование/категоризация (25%).

Состав команды: 5 человек (см. раздел ниже).

Команда из 5 — кто что делает

Впишите имена напротив ролей и работайте строго по своему контуру. Пересечения только на стыках API ↔ UI.

#	Роль	Имя	Главный результат
1	Product + питч	_____	Механики + презентация + доки
2	Backend lead	_____	Auth, модели, тест, категории
3	Backend + match	_____	Подбор, ранжирование, приглашения
4	Frontend кандидат	_____	Весь кабинет кандидата
5	Frontend работодатель + DevOps	_____	Кабинет работодателя + Docker/деплой

Человек 1 — Product / механики / питч

Делает:

- Справочник: отрасли, специализации, грейды, категории (Backend × Junior/Middle/Senior и т.п.)
- Механика теста: банк задач, как выдаются варианты (антислив), как считается балл и грейд
- Механика подбора: формула ранжирования (тест + ФСП), текст «почему в подборке»
- 5 демо-вакансий + 30 синтетических кандидатов (описание для сидов)
- Процедура валидации: как проверяете качество (таблица «вакансия → топ кандидатов»)
- Документация (архитектура словами, тест, подбор, ФСП, валидация)
- Презентация + репетиция питча, ответы жюри

Не делает: не пишет основной код, только постановка и проверка сценариев с командой.

Готово, когда: на питче за 5 минут ясно «как тест» и «как подбор», в доках это же написано.

Человек 2 — Backend lead (ядро)

Делает:

- Проект FastAPI + PostgreSQL + структура модулей
- Модели и миграции: User, Candidate, Employer, Company, Category, Grade, TestAttempt
- Auth: регистрация/логин email, роли, пароли
- API: профиль кандидата, опрос, старт/сдача теста, присвоение категории, лимит смены грейда
- Сиды задач теста + демо-аккаунты
- OpenAPI/Swagger в порядке

Стык: отдаёт контракты API человеку 4 (кандидат) и человеку 3 (данные для match).

Готово, когда: кандидат через API проходит опрос → тест → получает категорию.

Человек 3 — Backend match + приглашения

Делает:

- API работодателя: компания, описание потребности
- Подборка кандидатов по категории + ранжирование + **reason** (обоснование)
- Поиск/фильтры: спец, грейд, стек, есть ли ФСП
- Приглашения: создать, статусы (отправлено / просмотрено / принято / отклонено)
- Правило: контакты скрыты до ассерт; зарплата **от-до** обязательна
- Заглушка ФСП achievements + кейс «истории нет»
- После MVP (если успеет): вакансии, отклики, регулярные задания

Стык: контракты для человека 5; использует категории/баллы от человека 2.

Готово, когда: работодатель через API получает подборку и шлёт приглашение, кандидат может ассерт/decline.

Человек 4 — Frontend кабинет кандидата

Делает экраны:

1. Регистрация / логин
2. Профиль / резюме (форма)
3. Опрос: отрасль → специализация → грейд
4. Тест (таймер/прогресс по желанию)
5. Результат: категория, грейд, история
6. Входящие приглашения: принять / отклонить, видна зарплата
7. Настройки: приватность, согласие 152-ФЗ, поле ФСП ID

Стык: только API человека 2 (+ приглашения от 3).

Готово, когда: жюри проходит весь путь кандидата без подсказок.

Человек 5 — Frontend работодатель + DevOps

Делает экраны:

1. Профиль компании

2. «Кого ищем» (потребность)
3. Подборка: категории → кандидаты → обоснование
4. Поиск / фильтры по банку
5. Форма приглашения (описание + зарплата от-до)
6. Статусы исходящих приглашений

DevOps:

- `Dockerfile` + `docker-compose.yml` (web, api, db)
- README: запуск
- Деплой прототипа **или** запись видео сквозного сценария
- Помощь с седами данных от человека 1

После MVP (если успеет): публикация вакансий, адаптив.

Готово, когда: жюри как работодатель находит категорию и приглашает; проект поднимается по инструкции.

Ежедневный синхрон (15 мин)

1. Человек 1: блокеры по механикам / демо-данным
2. 2↔4: API кандидата стыкуется?
3. 3↔5: API работодателя стыкуется?
4. Человек 5: деплой живой?
5. Общий: что нужно для сквозного демо сегодня вечером

Кто за что отвечает на питче

Вопрос жюри	Отвечает
Зачем обратная механика / LHH	Человек 1
Как устроен тест / антислив	Человек 1 (+ 2 по коду)
Как ранжируете	Человек 1 (+ 3 по коду)
Архитектура / запуск	Человек 2 и 5
Демо в UI	Человек 4 и 5 показывают

1. Что нужно сделать (обязательный MVP)

Без этого решение слабо оценивается. Делать в первую очередь.

1.1. Два кабинета

Роль	Обязательный функционал
Кандидат	Регистрация по email, профиль/резюме, опрос (отрасль + специализация), выбор грейда, прохождение теста, категория и грейд, входящие приглашения (принять/

Роль	Обязательный функционал
	отклонить), настройки приватности + связь с ФСП ID (заглушка ок), обработка «нет истории ФСП»
Работодатель	Профиль компании, описание потребности (стек, спец, грейд), подборка кандидатов с обоснованием, поиск/фильтры по базе, приглашение с описанием и зарплатой «от-до» в Р, статусы приглашений

1.2. Ключевые механики (смысл конкурса)

1. **Подбор:** кандидат попадает в категорию только после опроса + теста (не по самоописанию резюме). Внутри категории — ранжирование (тест + достижения ФСП). Работодатель видит категорию → кандидата → шлёт приглашение.
2. **Тестирование:** свой подход, устойчивый к «сливу» заданий. Не один общий набор на всех. Грейд не понижается принудительно: можно пересдать ниже/выше, но не чаще 1 раза в N месяцев (например, 3).
3. **Видимость:** контакты кандидата скрыты, пока он не принял приглашение. Зарплата всегда видна в приглашении/вакансии.

1.3. Сквозное демо (обязательно показать жюри)

Регистрация кандидата

- опрос + выбор грейда + тест
- присвоение категории
- работодатель описывает потребность
- подборка / поиск категории
- приглашение конкретному человеку (с зарплатой)
- кандидат принимает / отклоняет

Отдельно объяснить на питче: **как устроен тест** и **как устроен подбор**.

1.4. Сдача

- Репозиторий с открытым кодом
- Презентация (pptx/pdf)
- Задеплоенный прототип **или** видео + инструкция запуска
- Документация: архитектура, механика теста, механика подбора, валидация, схема интеграции с ФСП, OpenAPI, версии библиотек, инструкция запуска
- Docker Compose — желательно (плюс к оценке)

2. Что можно сделать (желательно / плюсы)

Делать **после** рабочего MVP. Даёт баллы в «доп. функционал» (7%) и усиливает питч.

Фича	Зачем
Отклик кандидата на вакансии + публикация вакансий работодателем	Классический сценарий «пока нет приглашений»
Регулярные короткие задания от работодателей	Актуальность профиля
Парсинг PDF-резюме + генерация PDF-профиля	Удобство + плюс за NLP
ML/NLP для разбора резюме, категорий, подбора	Прямо указано как преимущество
Защита от фиктивных вакансий (концепция + простой прототип)	За рамками MVP, но плюс
Концепция интеграции с ATS	Архитектурный плюс
Keycloak / совместимость с ФСП ID	Упростит «путь после хакатона»
Адаптив под мобильные	Опционально для MVP
Микросервисы	Не обязательны; нормальный модульный монолит ок

3. Рекомендуемый стек (для скорости)

Слой	Технология
Frontend	TypeScript + Next.js (или React)
Backend	Python + FastAPI
БД	PostgreSQL
Auth	JWT / сессии (Keycloak — если успеете)
API docs	OpenAPI (Swagger) из FastAPI
Деплой	Docker Compose + любой хостинг

Стек можно менять — в ТЗ ограничений нет. Главное: Frontend / Backend / БД разделены, код открытый, запуск по инструкции.

4. Разделение по ролям в команде

Ниже — роли под типичную команду **4–5 человек**. Если человек меньше — совмещать соседние роли сверху вниз по приоритету.

Роль А — Product / механики (1 человек) — **критично для победы**

Ответственность: смысл продукта, критерии 35% + 25%.

Сделать:

- ☐ Описать категории (специализация × грейд), справочник грейдов

- ☐ Спроектировать механику теста (антислив: банк вариантов / параметризация / пул задач по темам)
- ☐ Спроектировать ранжирование внутри категории + текст «почему в подборке»
- ☐ Набросать 3–5 вакансий и тестовые профили кандидатов
- ☐ Описать процедуру валидации (как сами проверяете качество подбора и теста)
- ☐ Текст для документации: механика теста, механика подбора, валидация, интеграция ФСП

Не делает: вёрстку и низкоуровневый код, если есть отдельные разработчики — только постановка и проверка сценариев.

Роль В — Backend (1–2 человека)

Ответственность: API, бизнес-логика, БД, безопасность.

Сделать (MVP):

- ☐ Модели: User, Candidate, Employer, Company, Category, Grade, Test, Invitation, Vacancy (опц.)
- ☐ Auth: регистрация/логин по email, роли, хеш паролей
- ☐ API кандидата: профиль, опрос, старт/сдача теста, категория, приглашения, настройки, ФСП ID
- ☐ API работодателя: компания, потребность, подборка, поиск/фильтры, приглашения + статусы
- ☐ Правила: зарплата обязательна; контакты скрыты до ассерт
- ☐ Лимит смены грейда (например, раз в 90 дней)
- ☐ OpenAPI/Swagger
- ☐ Сиды: демо-пользователи, категории, задачи теста

После MVP (если успеете):

- ☐ Парсинг PDF резюме
 - ☐ Регулярные задания
 - ☐ Отклики на вакансии
 - ☐ Заглушка/адаптер ФСП achievements
-

Роль С — Frontend (1–2 человека)

Ответственность: два кабинета, UX (критерий 10%), демо-путь.

Сделать (MVP):

- ☐ Роутинг: кандидат / работодатель
- ☐ Экраны кандидата: регистрация, профиль, опрос, тест, результат/категория, приглашения, настройки
- ☐ Экраны работодателя: компания, «кого ищем», подборка с обоснованием, поиск, форма приглашения (зарплата!), статусы
- ☐ Состояния: loading / empty / error; понятный путь без пояснений жюри
- ☐ Цвета в гамме ФСП (брендбук), аккуратный UI без перегруза

После MVP:

- ☐ Список вакансий + отклик

- ☐ Публикация вакансий
 - ☐ Адаптив
-

Роль D — DevOps / данные / демо (можно совмещать с Backend)

Сделать:

- ☐ Dockerfile + docker-compose.yml (api, web, db)
 - ☐ README: запуск за 3–5 команд
 - ☐ Выкладка прототипа (Railway / Render / VPS / etc.)
 - ☐ Видео-запись сквозного сценария (запасной план)
 - ☐ Синтетические данные: 20–50 кандидатов, 5–10 работодателей, достижения ФСП у части людей
-

Роль E — Питч и документация (можно совмещать с Product)

Сделать:

- ☐ Презентация: проблема → обратная механика → тест → подбор → демо → валидация → roadmap
 - ☐ Сопроводительная документация (обязательна)
 - ☐ Репетиция питча (критерий 10%)
 - ☐ Ответы на вопросы: «почему тест не сливается», «как ранжируете», «что с ФСП без API»
-

5. Приоритеты работ (порядок)

День / этап 1 — фундамент

1. Согласовать категории, грейды, формулу ранжирования, схему теста
2. Поднять Backend + БД + auth + сиды
3. Каркас Frontend двух кабинетов

День / этап 2 — сердце продукта

4. Полный флоу теста → категория
5. Подборка + поиск + приглашение + статусы
6. Сквозное демо на локалке

День / этап 3 — сдача и плюсы

7. Docker, деплой, README, OpenAPI
 8. Документация + валидация на тестовом наборе
 9. Презентация + видео
 10. Только потом — отклики, PDF, регулярные задания, ATS-концепт
-

6. Definition of Done (готово = можно сдавать)

- ☐ Кандидат проходит тест и получает категорию

- ☐ Работодатель видит категории/кандидатов и понимает, почему человек в топе
- ☐ Работодатель шлёт приглашение **с зарплатой**
- ☐ Кандидат принимает/отклоняет; до ассерт контакты скрыты
- ☐ Кандидат без ФСП-истории обрабатывается корректно
- ☐ Есть репозиторий, доки, презентация, запуск по инструкции (или деплой)
- ☐ На питче за 3–5 минут объясняется тест + подбор

7. Чего не делать в ущерб MVP

- Красивый микросервисный зоопарк вместо работающего монолита
- Идеальный Keycloak вместо простого auth
- Парсинг всех сайтов вакансий (в ТЗ прямо: не парсер чужих площадок)
- Огромный банк задач «на всё ИТ» — достаточно 1–2 направлений (например, Backend + Frontend) для демо
- Полировка UI до потери механик

8. Краткая шпаргалка по баллам

Критерий	Вес	Кто тянет
Новая механика подбора	35%	Product + Backend + Frontend
Тест, валидация, категоризация	25%	Product + Backend
UI/UX	10%	Frontend
Техника	7%	Backend + DevOps
Полнота + доки	6%	Все + Product
Доп. функционал	7%	После MVP
Питч	10%	Product / спикер

Вывод для команды: сначала «работодатель приглашает категоризированного кандидата после теста», потом всё остальное.